Everything You Need for the 4pm Meeting

The whole project explained slowly, from the ground up:
what we did, what we found, and what the meeting will be about

Friday, 31 July 2026

How to read this

This document assumes nothing. It starts with what a simulation study even is, builds up every idea one at a time, and only then walks through our results and the meeting agenda. Read it in order. Sections 1–6 are the background; Sections 7–9 are what we did and found; Section 10 is the meeting itself. If you are short of time, read the boxes.

Contents

1 What is a simulation study?

Imagine you want to test whether a new set of scales weighs things correctly. The best test is simple: take an object whose weight you already know exactly, say a 1 kg reference weight, put it on the scales many times, and see what they say. If they say 1.02 kg on average, the scales are *biased*. If the readings jump around a lot, the scales are *noisy*.

A simulation study does exactly this, but for *statistical methods* instead of scales. The problem with testing a statistical method on real data is that in real data **nobody knows the right answer**, so you can never tell whether the method got it right. The solution: **invent the data yourself**.

Here is the recipe, and it is the recipe our whole project follows:

1. **Write down rules for a small artificial world.** For example: people have an age drawn at random between 40 and 75; older people have higher cholesterol; people with higher cholesterol are more likely to get a drug; the drug lowers the risk of a bad outcome by a known amount. These rules are called the *data-generating mechanism*.
2. **Because you wrote the rules, you know the true answer exactly.** In our world, the drug truly lowers the risk of the outcome by 10.8 percentage points. No method can know this; we know it because we built the world.
3. **Generate many datasets from those rules.** A computer rolls the dice and produces a fake "study" of 3,000 people. Then it does it again, and again — 1,000 times in our case. Each dataset is like a fresh study run in a parallel universe with the same rules but different random luck.
4. **Run the statistical method on every dataset** as if it were real data. The method does not get to see the rules or the true answer — only the data, exactly like a real analyst.
5. **Compare the 1,000 answers with the truth.** Is the method right on average? How much do its answers wobble? Are its claimed margins of error honest?

Why 1,000 datasets and not just one?

One dataset could give a lucky or unlucky answer by chance. With 1,000 repeats, luck averages out, and we can see the method's true habits: where its answers centre, and how widely they spread. It is the "weigh the reference weight many times" idea.

2 The causal question our artificial world poses

2.1 The two-worlds idea

The methods being tested all try to answer a *cause-and-effect* question: **how much does taking a statin (a cholesterol drug) change the risk of a bad health outcome?**

Think of every person as having two possible futures:

- Y^1: what would happen to them **if they took the statin**;
- Y^0: what would happen to them **if they did not**.

Each is either 1 (the bad outcome happens) or 0 (it does not). If we could see both futures for everyone, cause and effect would be easy. The three quantities the project talks about constantly are just averages of these futures over the whole population:

Symbol	In words	True value in our world
$E(Y^1)$	average risk if *everyone* took the statin	0.234 (23.4%)
$E(Y^0)$	average risk if *nobody* took the statin	0.342 (34.2%)
ATE $= E(Y^1) - E(Y^0)$	the change in risk caused by the drug	-0.108

So in our artificial world the statin truly cuts the risk from about 34% to about 23% — a reduction of 10.8 percentage points. Every method is trying to recover these numbers from data.

2.2 The fundamental problem

In any real dataset (and in our fake ones, on purpose), **you only ever see ONE of the two futures per person**: the one matching what they actually did. If Maria took the statin, you see her Y^1 and her Y^0 is forever missing; for Tom, who did not take it, you see Y^0 and Y^1 is missing. Half the information is always gone. Estimating $E(Y^0)$ means filling in the missing "no-statin" future **for all the people who did take it** — and that is where statistical methods come in.

2.3 Confounding: why you cannot just compare groups

Why not simply compare the outcomes of people who took the statin with those who did not? Because in our world (as in real life) **doctors give statins to the sicker people**: those with higher cholesterol, diabetes, higher risk scores. So statin-takers were already at higher risk to begin with. Comparing the two groups directly mixes up the effect of the drug with the fact that its takers were sicker. This mixing is called **confounding**, and the variables doing the mixing (age, cholesterol, diabetes, risk score) are **confounders**.

All six methods below are different strategies for undoing this mixing, using the confounders that the data records for every person.

3 The six methods, one at a time

Every method needs to estimate one or both of these helper quantities (the project calls them *nuisance models* — needed on the way, not interesting in themselves):

- the **outcome model**: given a person's confounders, what is their risk of the outcome? (One version for the treated, one for the untreated.)
- the **propensity score**: given a person's confounders, what was their probability of receiving the statin?

3.1 G-computation (the plug-in)

Fit an outcome model, then use it to *predict the missing future* for everyone. To estimate $E(Y^0)$: predict every single person's risk as if untreated (the model fills in the missing Y^0 for the statin-takers), then average the predictions. Simple and intuitive — but it leans entirely on the outcome model's predictions being right. **It has no built-in correction if they are slightly wrong.** This is the method the whole project ends up scrutinising.

3.2 IPW (inverse probability weighting)

Instead of modelling outcomes, reweight people. The untreated people who *almost got* the statin (high propensity score) are rare but stand in for the many similar people who did get it, so they receive a big weight. Averaging the reweighted untreated outcomes estimates $E(Y^0)$. Leans entirely on the propensity model.

3.3 AIPW (augmented IPW) — doubly robust

Combines both: start from the outcome model's prediction, then use the weighted residuals (how wrong the model was for the people we can check) to correct it. If *either* the outcome model *or* the propensity model is right, the answer is right — hence "doubly robust". Crucially, this correction also cancels most of the small systematic errors machine learning makes.

3.4 TMLE (targeted maximum likelihood)

Same goal as AIPW, different route: it *nudges the outcome model's predictions* just enough to remove the bias, then averages the nudged predictions. Doubly robust as well. In our results AIPW and TMLE behave almost identically — expected, as they are two routes to the same theory.

3.5 Cross-fitting (the "SC-" versions)

A safety measure on top of AIPW/TMLE. Problem: if a flexible model is fitted on the same people it then makes predictions for, it has partly *memorised* them, and its errors are deceptively small. Cross-fitting splits the data into 5 folds; each person's predictions come from models trained on *other* folds only. Nobody's estimate uses a model that saw them. "SC-AIPW" and "SC-TMLE" are AIPW and TMLE with this safeguard.

4 Machine learning, and why it causes the trouble

4.1 The super learner

Instead of assuming a formula for the nuisance models ("risk rises linearly with age…"), let an algorithm learn the shape from data. The paper uses a **super learner**: it takes six candidate methods — ordinary logistic regression, two smooth-curve models (GAMs), a **random forest** of 500 decision trees, a small neural network, and the plain average — tests each by cross-validation, and blends them with weights that favour whichever predicts best. Flexible and sensible; widely recommended.

4.2 The catch: plug-in bias

Flexible methods buy their flexibility by *smoothing*: to avoid chasing noise, they pull predictions toward typical values. That produces many small, systematic errors — especially for unusual people at the edges of the data. Now recall what g-computation does with those predictions: it *averages* them. Small systematic errors in the same direction do not cancel when averaged — they accumulate into a stubborn **bias** in the final answer. And no confidence interval can warn you, because the interval only measures wobble, not being systematically off-target. This is "plug-in bias" (Shaun also calls it *slow convergence*). The theory says AIPW/TMLE's correction term removes it, and cross-fitting removes the memorisation part. **The paper's purpose — and ours — is to demonstrate all of this happening with actual numbers.**

5 How we judge a method: the report card

For each method we have 1,000 answers (one per dataset) and the truth. From these:

- **Bias**: average answer minus truth. Zero is perfect. Bias of $+0.026$ on a truth of -0.108 means the method typically reports about -0.082 — it understates the drug's benefit by a quarter.
- **ESE (empirical standard error)**: the spread of the 1,000 answers — how much the method wobbles from study to study. This is measured directly, so it is the "true" uncertainty.
- **ASE (average claimed standard error)**: the uncertainty the method *claims* for itself. If ASE $\approx$ ESE, the method is honest about its own precision.
- **Coverage**: each dataset also produces a 95% *confidence interval* — a range that should contain the truth. Coverage asks: out of 1,000 intervals, how many actually contained the truth? **An honest method scores about 95%.** A method scoring 64% is misleading its user in one study out of three — and the user has no way to tell.

One convention to remember (Shaun designed it)

G-computation cannot compute its own standard error (no formula exists for it with arbitrary machine-learning models; the paper used a very expensive re-sampling trick called the bootstrap). Our solution, chosen by Shaun: build its intervals from the **empirical** SE — the directly measured spread. This has a beautiful consequence: those intervals have exactly the right *width*, so **if coverage still falls below 95%, the failure can only be bias** — the intervals sit in the wrong place. It isolates the disease from the symptoms.

6 The three "specifications": easy, wrong, and realistic

Every study runs each method three times, differing only in how the nuisance models are built:

- **True**: we cheat — the models are given the exact mathematical form used to generate the data. Everything should work; this is the sanity check.
- **Main-effects**: deliberately too-simple formulas (straight lines only). Represents a careless analyst; methods should show bias.
- **Machine learning**: the honest modern approach — no formula assumed, the super learner learns the shape. **This is the case the project is about.**

7 The story of the project, phase by phase

7.1 Phase 1: try to reproduce the paper exactly

We took the authors' published code and ran it **without changing a single line** (their files stay untouched to this day; anything we needed lives in our own separate scripts). Modern software broke it in three places, so we built a time-capsule computing environment with 2019-era library versions where it runs as-is. Their own test example then passed completely, and the runs that were feasible on a laptop all matched the paper's behaviour. But the full study needed about 30,000 hours of computing — months — because the paper re-fits the super learner up to 1,456 times per dataset. The authors used a computing cluster; we did not have one, and Shaun did not want one ("I don't want to fry the planet").

7.2 Phase 2: the agreed lighter design, and the reproduction

Shaun redesigned the study to ask the same question 50 times cheaper: drop the expensive bootstrap (use the empirical-SE convention above), and replace the paper's exotic "double" cross-fitting with standard 5-fold cross-fitting. We ran all six methods on 1,000 datasets (about 4 days of laptop computing) and reproduced the paper's message. The key table (coverage; should be 95%):

Estimator	True	Main-effects	Machine learning
G-computation	94.8%	76.8%	**64.0%**
IPW	94.4%	86.7%	93.7%
AIPW	94.1%	85.4%	90.8%
TMLE	93.8%	85.9%	90.1%
SC-AIPW	95.1%	90.4%	93.5%
SC-TMLE	94.6%	92.1%	93.8%

Read it row by row: with the true models (column 1) everyone behaves — the sanity check passes. With machine learning (column 3), **g-computation collapses to 64%** while the doubly-robust methods hold up and the cross-fit versions are best. That IS the paper's message, reproduced. Two validation points to remember: g-computation's machine learning bias came out at

+0.026, **matching the paper's published value exactly**, and the parametric rows match the paper almost digit for digit.

7.3 Phase 3: where does the bias live? (the best finding)

Shaun then declared the doubly-robust story settled and pointed everything at g-computation — and at a sharper question: is the bias in $E(Y^1)$, in $E(Y^0)$, or both? We ran the original-style g-computation recording both means, plus three variants changing one thing at a time. All numbers below are for the machine-learning specification, shown as bias (coverage):

	$E(Y^1)$	$E(Y^0)$	ATE
Baseline (one outcome model, as the paper)	+0.014 (82.7%)	−0.013 (75.1%)	+0.026 (64.0%)
Study 1: random forest alone	+0.025 (61.0%)	−0.014 (66.9%)	+0.039 (**30.0%**)
Study 2: half the sample (n=1500)	+0.023 (78.5%)	−0.014 (82.8%)	+0.037 (59.8%)
Study 3: two separate outcome models	−0.001 (**95.8%**)	−0.017 (**68.6%**)	+0.017 (85.3%)

Walk through it slowly:

- **Baseline row**: the two means are biased in *opposite* directions ($E(Y^1)$ pushed up, $E(Y^0)$ pushed down). The ATE subtracts one from the other, so the two biases **add**: $+0.014 - (-0.013) = +0.026$. The ATE number was hiding this structure entirely.
- **Study 1**: using the random forest *alone* (no blending) makes everything much worse — ATE coverage crashes to 30%. The super learner's other members were reining the forest in. (Also kills the idea that the neural network was the culprit.)
- **Study 2**: with half the data, bias grows. That is "slow convergence" made visible: less data, slower approach to the truth, bigger leftover bias.
- **Study 3** (the paper fits ONE outcome model with treatment as an input; here we fit two separate models, one per arm): this **completely fixes** $E(Y^1)$ — but $E(Y^0)$ stays biased at 68.6% coverage.

The project's best sentence

Under machine learning, g-computation's problem is not spread evenly: after separating the models, **essentially all of the remaining bias sits in $E(Y^0)$ — the average outcome if nobody were treated — while $E(Y^1)$ is estimated almost perfectly.** This is why Shaun's new studies look at $E(Y^0)$ only.

8 What is running right now (studies 3c, 4, 5)

Shaun's current request, launched last night, finishing over the weekend. All three: estimate $E(Y^0)$ **only**, now with *all six* methods, under one strict rule: **no person who took the statin may be used to fit the untreated-outcome model** (the paper's single-model approach broke this rule, and that contamination is one suspect for the bias). The treated-arm model is never fitted at all — it is not needed.

To keep it cheap and comparable, each dataset's nuisance models are **fitted once and shared** by all six methods: one propensity model and one untreated-outcome model on the full sample, plus one of each in every fold for the cross-fit versions — 12 machine-learning fits per dataset, exactly Shaun's own count.

- **Study 3 continued**: $n = 3000$, super learner. Running now (∼1.5 days).
- **Study 4**: same but $n = 1500$ (∼1 day).
- **Study 5**: same but random forest instead of super learner (∼2 hours).

Because no software package computes these arm-specific versions, we wrote the estimators ourselves — and before launching anything, **checked them against the authors' own code on identical inputs: agreement to 16 decimal places**. The very first datasets hint that the doubly-robust methods repair the $E(Y^0)$ bias (preliminary — do not quote numbers yet).

9 The 4pm meeting

9.1 What Shaun has promised to explain

1. **Why he cares about $E(Y^1)$ and $E(Y^0)$ separately rather than the ATE** — "I'll explain why when we meet." Our results gave his instinct strong support (the ATE was hiding everything).
2. **Why "we are spared the algebra"** — he had planned a mathematically derived variant of study 3, and says our results made it unnecessary.
3. Likely also: the **11 August presentation** (re-read his email with the details beforehand) and **what comes after** studies 3c/4/5.

Have this explanation in mind — but let him say it first

Why would $E(Y^0)$ be the hard one? The untreated-outcome model is *trained* on untreated people, but its predictions get *averaged over everyone* — including the statin-takers, who are systematically sicker (that is the confounding). So the model must predict in regions where it saw little training data: an **extrapolation** problem, exactly where cautious, smoothing-prone machine learning is at its worst. If his explanation differs, take his.

9.2 Things worth raising yourself

- **Cross-fitting detail**: Zivich's package trains each fold's nuisance models on *one* other fold (600 people), not on the other four folds (2,400) as in the textbook version. Same number of fits, still valid, but less data per model. Our new studies follow his package's convention — does he ever want the textbook version?
- **A brittleness we found**: the authors' super learner occasionally crashes (its weight-optimiser fails; their internal folds are never shuffled). Our runs retry with reshuffled rows; if a dataset still fails, that one nuisance fit falls back to logistic regression — loudly logged, a handful of fits in twelve thousand. One sentence of transparency.
- **Timing**: study 3c is ∼1.5 days (slightly more than first guessed); all three still finish over the weekend.
- **The presentation**: should it lead with the reproduction, or with the $E(Y^0)$ finding?

9.3 Questions he might ask you, with honest answers

- *"Why do you trust your new estimator code?"* — Because before launching, it was checked against Zivich & Breskin's own code on identical inputs and agreed to machine precision; and the true-specification rows act as a permanent sanity check (they come out at ∼95% coverage).
- *"Are your results reproducible?"* — The datasets are fixed by the authors' own random seed; our analyses set a seed per dataset; everything (results, code, logs, decisions) is in the GitHub repository. Re-running gives identical numbers. (The paper's own ML results are not exactly reproducible — the authors set no seed for the estimation stage.)
- *"Why is IPW fine under machine learning here?"* — Its 93.7% owes partly to its conservative variance (its intervals are a little too wide, which flatters coverage); its bias (+0.011) is smaller than g-computation's but not zero.

- *"Is 1,000 datasets enough?"* — The chance wobble on a 95% coverage estimate is ±0.7 percentage points — far smaller than the effects we are seeing (64% vs 95%).

10 One-page cheat sheet

- Truths: ATE −0.108; $E(Y^1)$ 0.234; $E(Y^0)$ 0.342. 1,000 datasets; $n = 3000$; same datasets everywhere; zero failures.
- Paper's message reproduced: g-comp ML coverage **64%** (bias +0.026, matches paper exactly); DR methods ∼90–94%; cross-fit best.
- Arms: baseline biases +0.014/−0.013 (add to +0.026); two-model fix leaves **all bias in** $E(Y^0)$ (−0.017, 68.6%), $E(Y^1)$ clean (95.8%).
- RF alone: 30% coverage. Half data: bias +0.037. Both worse than baseline.
- Running: studies 3c/4/5 — $E(Y^0)$ only, all six estimators, untreated-only outcome model, shared nuisances (12 fits), done by Monday.
- Meeting: he explains the "why"; you raise cross-fit structure, SL brittleness, timing, presentation angle.

Every number here was recomputed from the raw per-dataset result files before this briefing was written.